Airflow operators

INTRODUCTION TO APACHE AIRFLOW IN PYTHON



Mike Metzger
Data Engineer

datacamp

Operators

- Represent a single task in a workflow
- Run independently (usually)
- Generally do not share information
- Various operators to perform different tasks

```
@dag(
  dag_id="Example_Dag"
)
def example_dag():
  @task
  def task1():
    return "The result from task1"

  task1()

example_dag()
```

@task (PythonOperator)

- Executes a Python function
- Any Python function can be decorated with @task
- Can pass data with other functions / tasks

```
from airflow.sdk import task
@task
def printme():
    print("This goes in the logs!")
printme()
```

@task arguments

- Can pass arguments to task / function like an ordinary Python function

```
@task
def printme(name: str):
    print(f"Hi {name} - This goes in the logs!")

printme(name='DataCamp')

# Adds:
#   Hi DataCamp - This goes in the logs!
# to the Airflow logs
```

@task.bash (BashOperator)

```
@task.bash
def bash_example():
  return "echo 'Example!'"

bash_example()
```

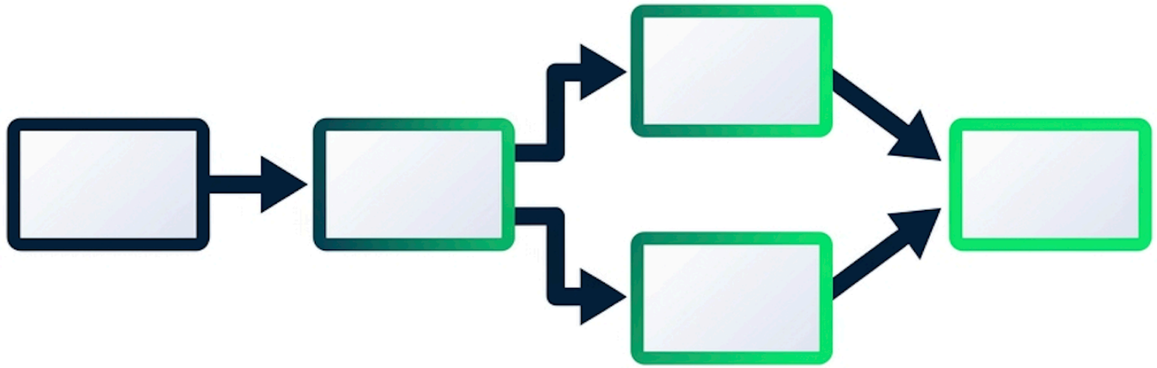
```
@task.bash
def run_cleanup():
  return "runcleanup.sh"

run_cleanup()
```

- Executes a given Bash command or script
- Runs the command in a temporary directory
- Can specify environment variables for the command

Task dependencies

- Each Dag has a set of tasks to complete
- Task dependencies specify running order
- Different methods to specify dependencies



Bitshift syntax

`>>` and `<<`

```
task1() >> task2()
# task1 completes before task2 starts

task1() >> task2() >> task3()
# task1 completes before task2 and task2 completes before task3

task1() >> task3()
task2() >> task3()
# task1 and task2 can run together but both must complete before task3 runs
```

Bitshift syntax example

```
# Download sales data before reconciling
download_sales_data() >> reconcile()

# Download inventory data before reconciling
download_inventory_data() >> reconcile()
```

- Both must complete before reconciling, but order isn't specified

Let's practice!

INTRODUCTION TO APACHE AIRFLOW IN PYTHON

Scheduling Dags

INTRODUCTION TO APACHE AIRFLOW IN PYTHON



Mike Metzger
Data Engineer

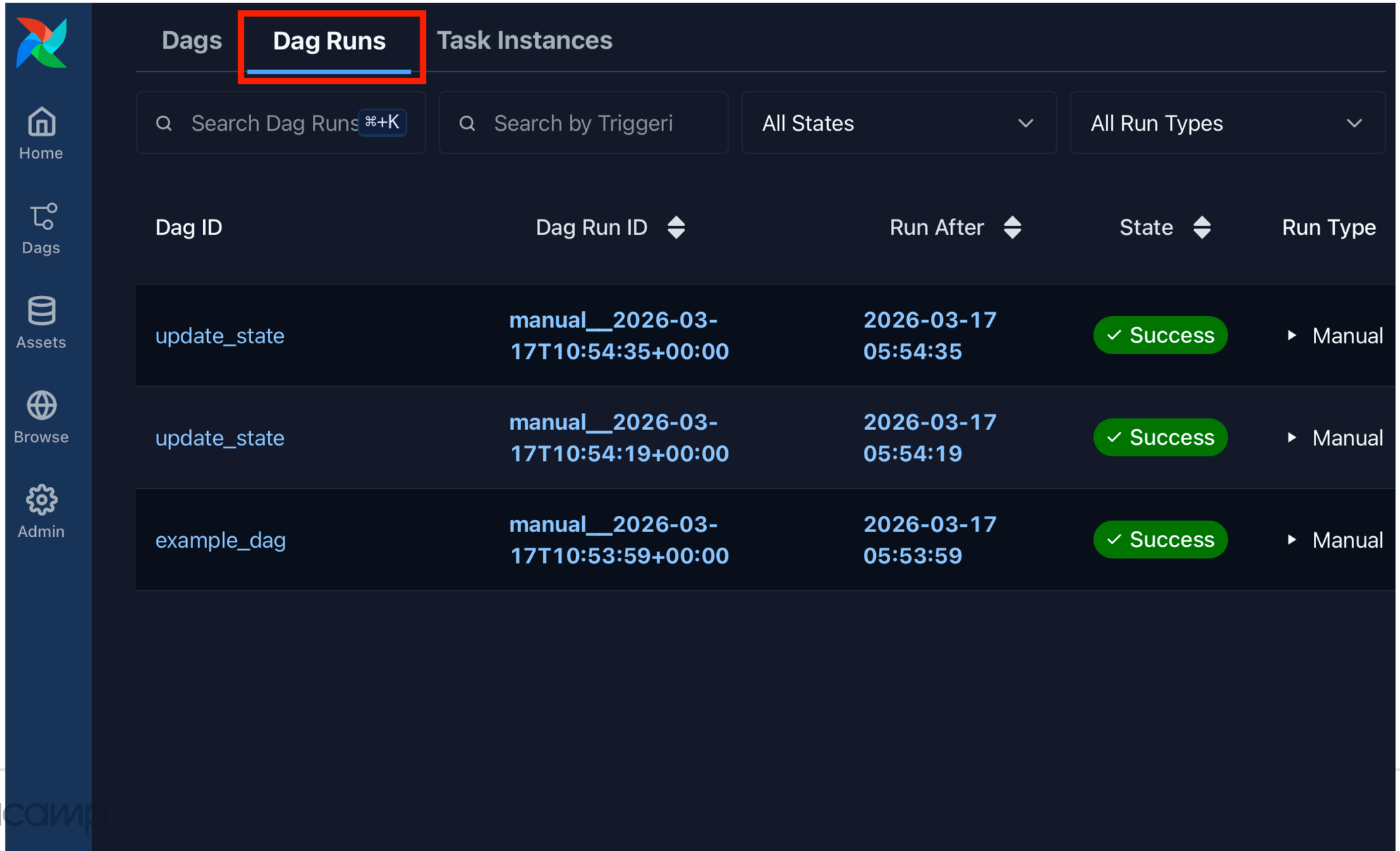
datacamp

Dag Runs

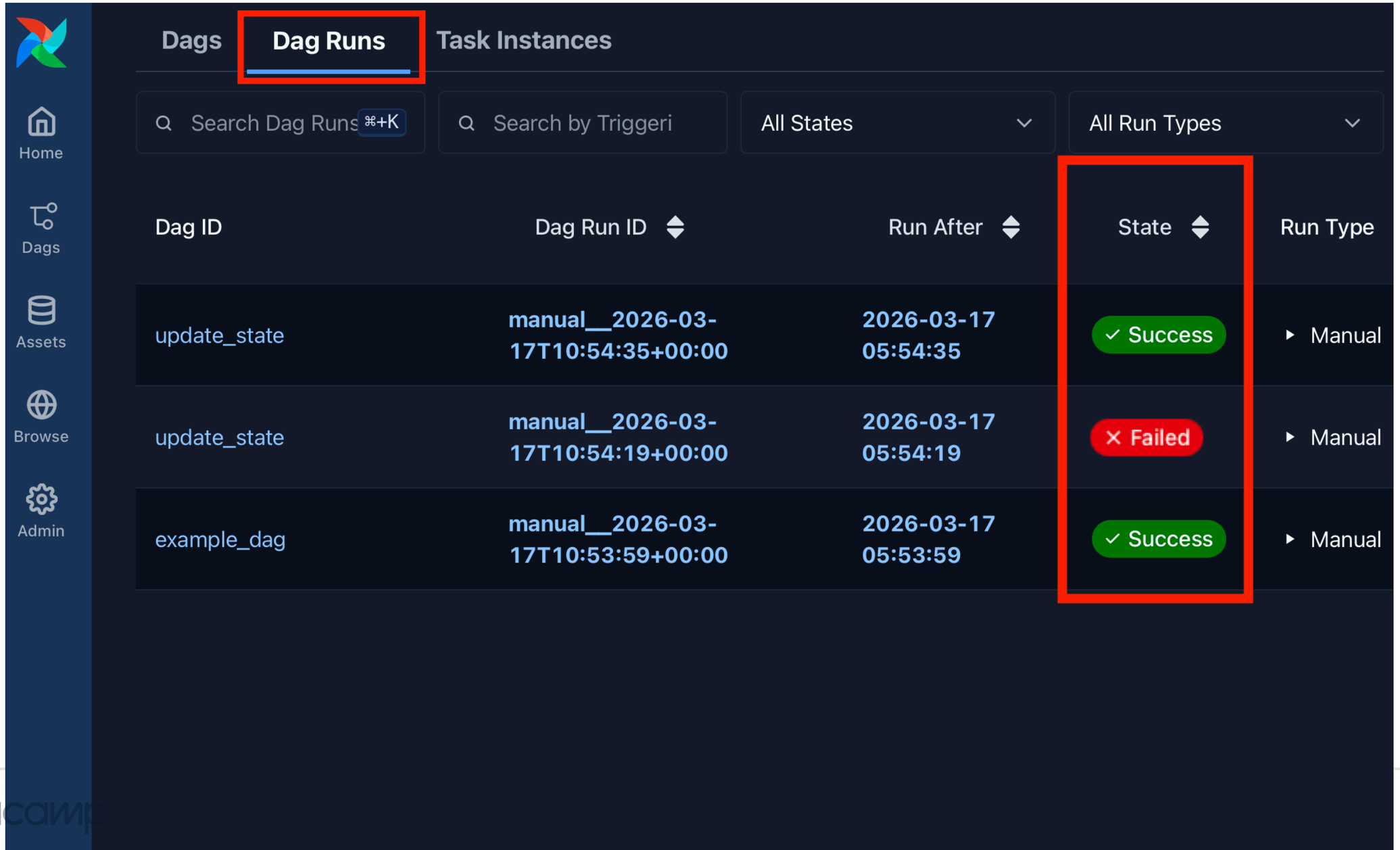
- A specific instance of a workflow at a point in time
- Can be run manually or via `schedule`
- Maintain state for each workflow and the tasks within
  - `running`
  - `failed`
  - `success`

[1] https://airflow.apache.org/docs/stable/scheduler.html

Dag Runs view



Dag Runs state



Schedule details

When scheduling a Dag, there are several attributes of note:

- `start_date` - The date / time to initially schedule the Dag run
- `end_date` - Optional attribute for when to stop running new Dag instances
  - `start_date` and `end_date` both use a `datetime(year, month, day)` object, such as:

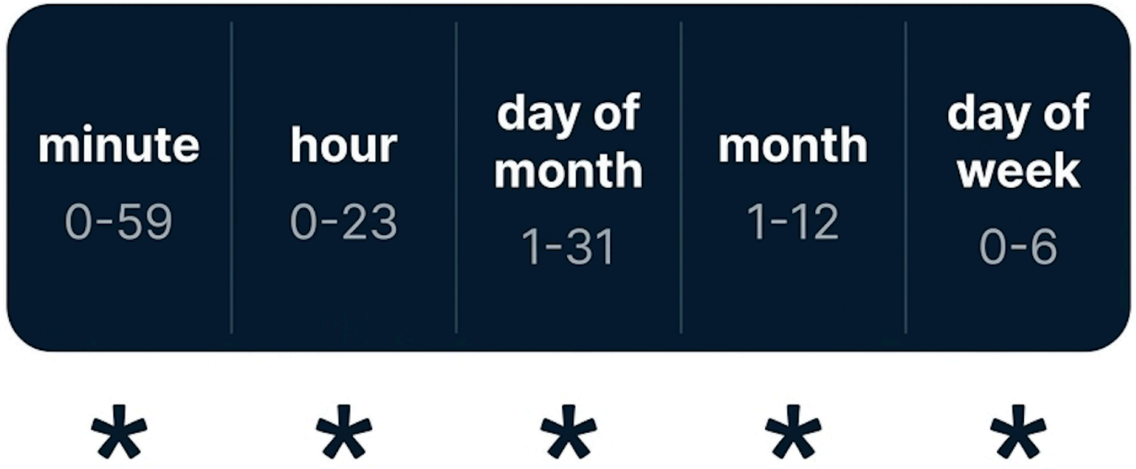
```
from pendulum import datetime
start_date=datetime(2026, 4, 10, tz="UTC")
```

Schedule

`schedule` represents:

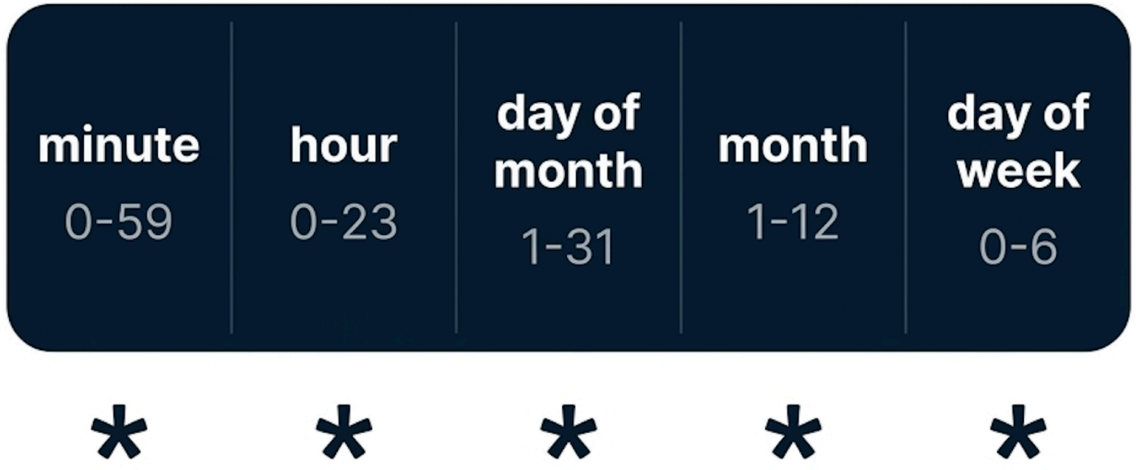
- How often to schedule the Dag
- Between the `start_date` and `end_date`
- Can be defined via `cron` style syntax, built-in presets, or timedeltas.

cron syntax



- From the Unix cron format
- 5 fields separated by a space
- `*` represents running for every interval (ie, every minute, every day)
- Can be comma-separated values in fields for a list of values

cron examples




```
0 12 * * *                # Run daily at noon
```

```
* * 25 2 *                # Run once per minute on February 25
```

```
0,15,30,45 * * * *       # Run every 15 minutes
```

Airflow scheduler presets

Preset:	cron equivalent:
@hourly	`0 * * * *`
@daily	`0 0 * * *`
@weekly	`0 0 * * 0`
@monthly	`0 0 1 * *`
@yearly	`0 0 1 1 *`

[1] https://airflow.apache.org/docs/stable/scheduler.html

Special presets

Airflow has three special `schedule` presets:

- `None` - Don't schedule ever, used for manually triggered Dags
- `@once` - Schedule only once
- `@continuous` - Run immediately after previous run finishes

timedelta

- Can also use `pendulum.duration`
- `duration(hours=6)`
- `duration(minutes=30)`

```
from pendulum import duration


@dag(
  dag_id="example_dag"
  schedule=duration(days=2)
)
```

Applying schedules

- Schedule defined on the Dag
- Uses the `schedule` parameter:

```
@dag(
  dag_id="example_dag",
  schedule="0 12 * * *"
)
```

```
@dag(
  dag_id="example_dag",
  schedule="@daily"
)
```

schedule issues

When scheduling a Dag, Airflow will:

- One full interval must pass beyond the start date
- Schedule the task at `start_date` + `schedule`

```
'start_date': datetime(2026, 2, 25, tz="UTC")
'schedule': @daily
```

Earliest start time to run the Dag is February 26th, 2026

Let's practice!

INTRODUCTION TO APACHE AIRFLOW IN PYTHON

Passing data between tasks with XCom

INTRODUCTION TO APACHE AIRFLOW IN PYTHON



Mike Metzger
Data Engineer

datacamp

What is XCom?

- "Cross Communication"
  - Allows tasks to talk to each other
- Stored in the Airflow metadata database
  - Pass small amounts of data
  - Filenames, URI, row counts



What not to send via XCom

- Large files
- DataFrames
- Full databases
- Large images



Implementing XCom

- Many ways to use XCom
- We'll focus on the TaskFlow API
- Extension of what we've already done with @tasks

XCom example

```
@dag(dag_id='Example_XCom')
def example_xcom():
  @task
  def get_data():
    return data

  @task(multiple_outputs=True)
  def clean_data(sourcedata):
    return clean(sourcedata) # Example, not implemented

  clean_data(get_data())
example_xcom()
```

XCom dependencies

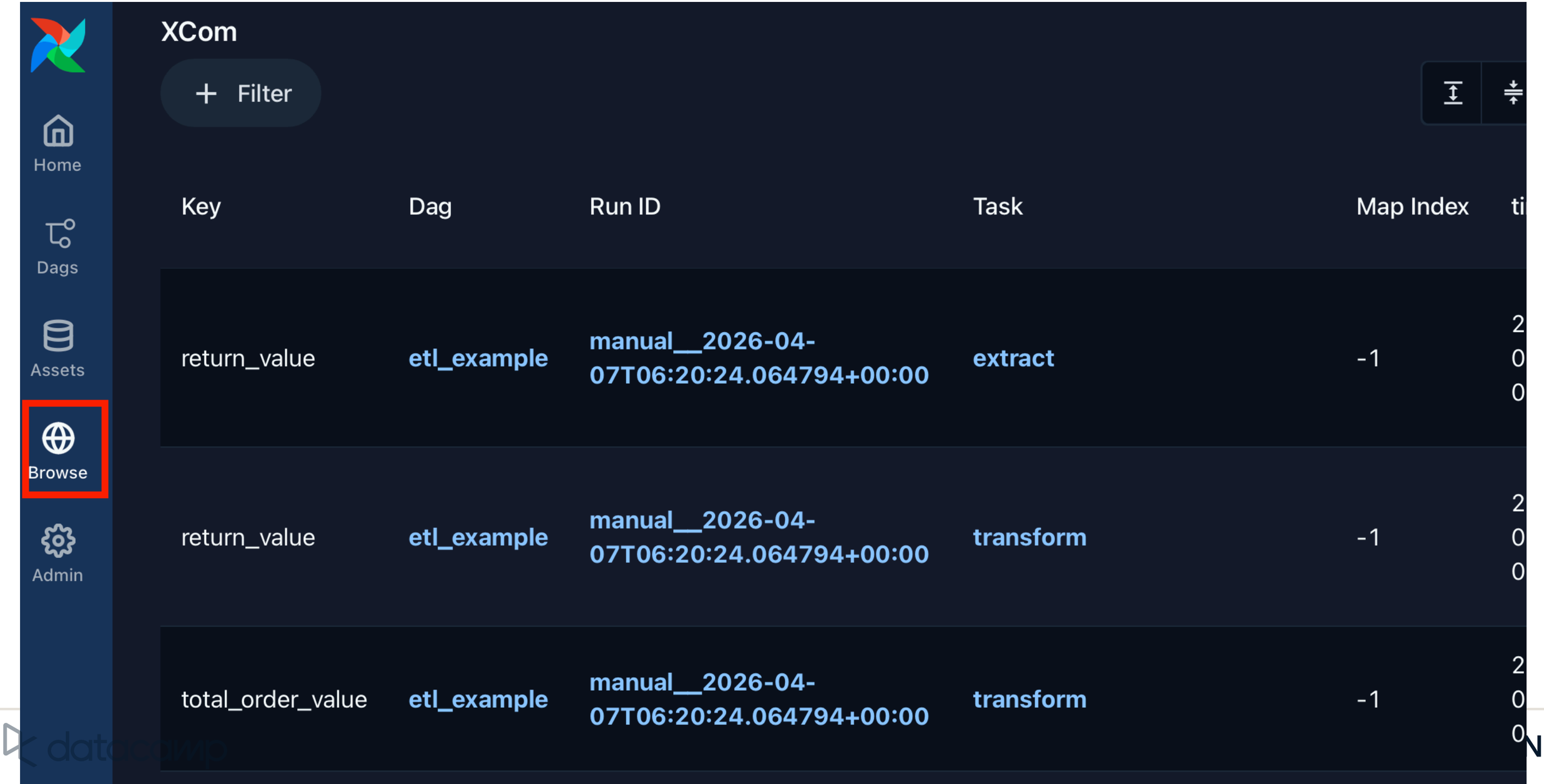
- XComs automatically define dependency order
- Example

```
clean_data(get_data())
```

- Conceptually the same as `get_data() >> clean_data()`
- Further example

```
result = clean_data(get_data())
result >> alert_when_complete()
```

Viewing XCom data



datacamp

Let's practice!

INTRODUCTION TO APACHE AIRFLOW IN PYTHON

Airflow Sensors

INTRODUCTION TO APACHE AIRFLOW IN PYTHON

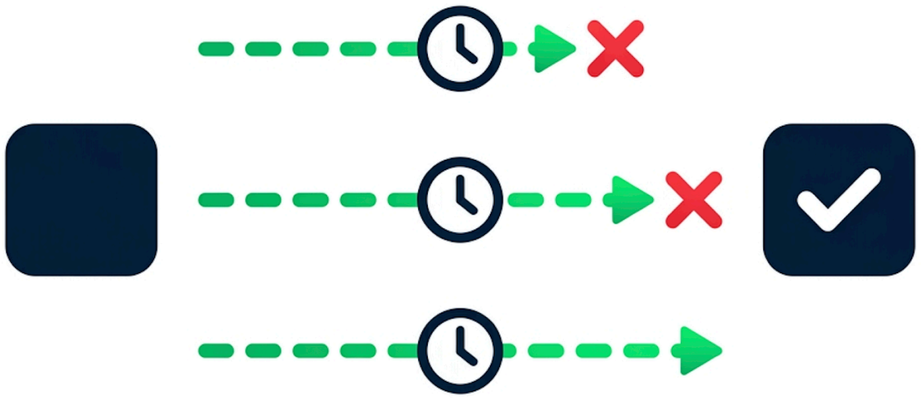


Mike Metzger
Data Engineer

datacamp

Sensors

- An operator that waits for a certain condition to be true
  - Create of a file
  - Upload of a database record
  - Response from a web request
- Can define how often to check for the condition to be true
- Assigned to tasks



Sensor details

- Derived from `airflow.sdk.BaseSensorOperator`
- Sensor arguments:
- `mode` - How to check for the condition
  - `mode='poke'` - The default, run repeatedly
  - `mode='reschedule'` - Give up task slot and try again later
- `poke_interval` - How often to wait between checks
- `timeout` - How long to wait before failing task
- Also includes normal operator attributes

File sensor

- Is part of the `airflow.providers.standard.sensors` library
- Checks for the existence of a file at a certain location
- Can also check if any files exist within a directory

```
from airflow.providers.standard.sensors.filesystem import FileSensor

file_sensor_task = FileSensor(task_id='file_sense',
                              filepath='salesdata.csv',
                              poke_interval=300,
                              timeout=3000
                              )

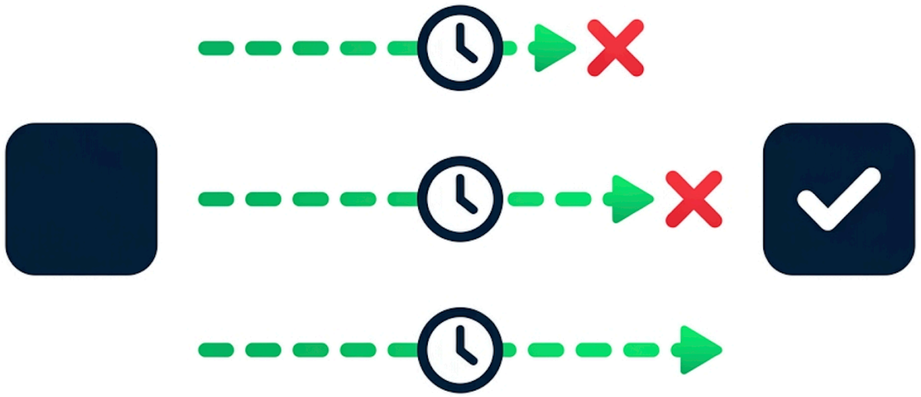
init_sales_cleanup() >> file_sensor_task >> generate_report()
```

Other sensors

- Many others in `airflow.providers.*.sensors`
- `ExternalTaskSensor` - wait for a task in another Dag to complete
- `HttpSensor` - Request a web URL and check for content
- `SqlSensor` - Runs a SQL query to check for content

When to use sensors?

- Uncertain when a condition will be true
- If failure not immediately desired
- Add task repetition without loops



Let's practice!

INTRODUCTION TO APACHE AIRFLOW IN PYTHON